



Towards accurate anomaly detection for cloud system via graph-enhanced contrastive learning

Zhen Zhang¹ · Zhe Zhu² · Chen Xu³ · Jinyu Zhang¹ · Shaohua Xu¹

Received: 14 May 2024 / Accepted: 11 October 2024 / Published online: 12 November 2024
© The Author(s) 2024

Abstract

As a critical technology, anomaly detection ensures the smooth operation of cloud systems while maintaining the market competitiveness of cloud service providers. However, the resource data in real-world cloud systems is predominantly unannotated, leading to insufficient supervised signals for anomaly detection. Moreover, complicated topological associations existed between cloud servers (e.g., computation, storage, and communication). While acquiring resource information, correlating the system topology is challenging. To this end, we propose the GCAD for cloud system anomaly detection, which integrates data augmentation, GraphGRU, contrastive learning, and reconstruction. First, GCAD constructs positive and negative sample pairs through the masking and Gaussian noise data augmentation. Then, the GraphGRU processes extended temporal graph data, extracting and fusing spatiotemporal features from resource status and system topology. In addition, GCAD introduces linear attention for encoding spatiotemporal representations to capture their global correlation information. The weight parameters of the encoder are optimized using a contrastive learning mechanism. Finally, GCAD utilizes a reconstruction technique to calculate anomaly scores, facilitating the evaluation of the state of the cloud system at each time point. Experimental results indicate that GCAD outperforms state-of-the-art compared methods on two real-world datasets that contain topology information.

Keywords Cloud computing · Anomaly detection · Graph neural network · Contrastive learning · Linear attention mechanism

Introduction

Amidst the rapid advancement of cloud computing technology, cloud systems have emerged as one of the pivotal infrastructures in modern society. However, as the scale and complexity of cloud systems continue to escalate, issues related to performance anomalies are becoming increasingly prevalent [1]. These anomalies manifest primarily as service delays due to resource contention and job failures caused by resource scarcity. The occurrence of anomaly events poses a significant threat to the stability and reliability of cloud systems, resulting in economic losses for service providers [2, 3]. Consequently, how to effectively detect anomalies in cloud systems has emerged as a pivotal research topic.

The significance of anomaly detection in cloud systems lies in its ability to identify abnormal behaviors, thereby optimizing resource allocation promptly. Service providers deploy monitoring tools to record crucial metrics of cloud systems [4], storing them as time series data [5]. These metrics include CPU usage, memory usage, network transmission speed, etc. By deeply analyzing the distributional

✉ Zhen Zhang
ZhenZ1992@outlook.com

Zhe Zhu
zhuzh@sdas.org

Chen Xu
xu_c20@fudan.edu.cn

Jinyu Zhang
jinyuz1996@outlook.com

Shaohua Xu
xush62@163.com

¹ School of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao 266590, China

² Shandong Artificial Intelligence Institute, Qilu University of Technology (Shandong Academy of Sciences), Jinan 250353, China

³ School of Computer Science, Fudan University, Shanghai 200433, China

characteristics of these multivariate time series, researchers can uncover usage patterns of cloud system resources, thereby identifying potential anomalies.

In cloud computing systems, traditional anomaly detection methods primarily involve constructing normal behavior patterns of the system and leveraging expert knowledge to formulate anomaly rule sets [6, 7]. While this approach is straightforward to implement, it has limitations in adapting to the dynamic changes in cloud system performance, and the setting of rules often lacks scientific data support. In contrast, deep learning models such as Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs) [8] offer significant advantages in capturing attributes and long-term dependencies of time series data. They are capable of extracting high-level feature representations from complex datasets [9, 10]. However, the actual performance data of cloud systems are characterized by high dimensionality and complexity [11], making it challenging for non-experts to discern their attributes directly. Consequently, such data tends to remain unannotated. Although deep learning is a data-driven approach, the lack of labeled data constrains the learning capability of the models. Moreover, anomaly events in cloud systems occur infrequently, and there may be unknown anomaly patterns. When applying deep learning models to anomaly detection tasks, the learning of anomalous features by the models is not sufficiently thorough.

To address these challenges, researchers have begun to employ reconstruction-based methods [12, 13] for detecting performance anomalies. The mainstream reconstruction approaches [14] construct feature representation models for normal sequences by minimizing the error between input data and their reconstructions [15]. However, cloud system performance data exhibit diverse characteristics due to varying resource types, which may result in significant differences among normal samples. Moreover, anomaly events manifest in various forms, such as subsequences anomalies and contextual anomalies. Training methods that depend solely on reducing reconstruction error may not adequately capture the high-level semantic information within the data. Unsupervised learning is extensively applied in data analysis to discern inherent structures and distributional properties [16]. This approach is characterized by its independence from externally furnished label information, relying instead on the statistical attributes and interrelationships of the data. Self-supervised learning represents a specialized variant within the domain of unsupervised learning. Unlike conventional unsupervised methods, it employs data preprocessing or transformation to create pseudo-labels for training. This strategy effectively mitigates the limitations of unsupervised learning in terms of interpretability and generalization [17]. Contrastive learning, as a self-supervised learning strategy [18], acquires effective data representations by contrasting the differences between different samples [19, 20]. It demon-

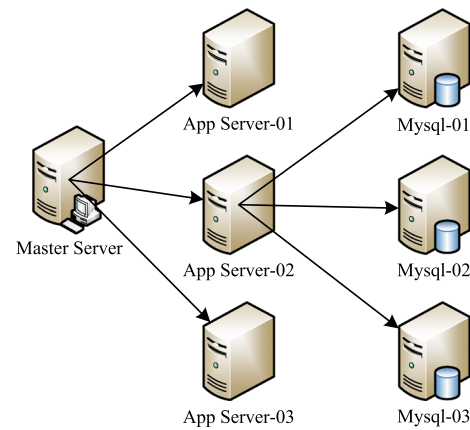


Fig. 1 Cloud system topology

strates robust feature extraction capabilities in unsupervised learning settings. Therefore, we integrate contrastive learning with reconstruction methods for anomaly detection in the cloud system. By mining and modeling the feature representations of normal behavior, we aim to identify and delineate performance anomalies.

Contrastive learning has achieved promising performance in anomaly detection tasks of the time series, but its application to cloud systems presents several challenges. These challenges arise from the dynamic nature of cloud system resource states over time and their strong correlation with the system's topological structure [21]. The topological structure of cloud systems reveals the differential correlations between servers, encapsulating the interactions and levels of interdependence among nodes concerning computation, storage, and communication capabilities, as illustrated in Fig. 1. The existing contrastive learning methods focus solely on temporal characteristics while neglecting spatial information when dealing with such data. To address this issue, we propose a graph-enhanced contrastive learning approach for detecting performance anomalies in cloud systems during runtime. This method analyzes cloud system performance data from both temporal and spatial dimensions. Initially, data augmentation is performed on the raw sequences in both the time and frequency domains to generate positive and negative sample pairs for contrastive learning. Subsequently, a combination of Graph Neural Networks (GNNs) and Gated Recurrent Units (GRUs) is employed to capture the integrated features of resource states and topological structures. During the training phase, we leverage a contrastive learning strategy to enhance the capability of the encoder to represent the features of positive samples. Building on this foundation, we utilize a decoder to reconstruct the input sequences and update the weight parameters of the decoder by minimizing the reconstruction error. During the testing phase, anomaly scores are calculated based on the reconstruction error. By comparing the anomaly scores at each time step against a

predefined threshold, we identify anomalous points within the sequence. The main contributions of this paper can be summarized as:

- We propose a method for detecting performance anomalies in cloud systems. It integrates contrastive learning and reconstruction strategies to establish representation models of normal data and distinguishes abnormal behaviors based on anomaly scores and thresholds.
- We propose a data augmentation strategy for time series. In the time domain, we apply geometric masks to the original sequences, generating positive samples while maintaining the temporal continuity of the sequences. In the frequency domain, we add Gaussian noise to the spectral components of the data, thereby diversifying the frequency content and broadening the spectral distribution.
- We propose a spatiotemporal information fusion model named GraphGRU by integrating GRU with GNN. This model achieves the fusion of cloud system topology information and resource status data.
- We employ a lightweight linear attention mechanism for feature extraction and representation, reducing computational complexity while maintaining model performance.

The remainder of the paper is organized as follows: “[Related work](#)” provides a comprehensive review of the current research status in time series and cloud system anomaly detection, and outlines the research approach of our proposed method; “[Methodologies](#)” describes the framework of GCAD and the implementation details of its main modules; “[Experiments](#)” validates the feasibility and effectiveness of the proposed method through experimental verification; “[Conclusion](#)” gives the conclusions and future work.

Related work

This section presents an overview of methods for identifying anomaly patterns in time series based on reconstruction and contrastive learning. Additionally, it discusses the latest advancements in detecting performance anomalies within cloud systems. A thorough synthesis and analysis of the associated research is provided to offer a comprehensive understanding of the current state and future directions in these domains.

Reconstruction based anomaly detection method for time series

In recent years, reconstruction-based time series anomaly detection has garnered significant attention within the data mining community. The core concept involves learning the

normal patterns of a time series and then reconstructing the original sequence to detect anomalies by calculating the reconstruction error. Addressing the anomaly detection problem for multi-sensor time series signals, Zhang et al. [22] introduced the Deep Convolutional Autoencoder-based Memory Network (CAE-M) algorithm. This approach leverages deep convolutional autoencoders to extract spatial dependencies and employs the Maximum Mean Discrepancy (MMD) to mitigate the impact of noise and anomalies on the model. Subsequently, the representations and reconstruction errors are fed into a memory network based on bidirectional Long Short-Term Memory (LSTM) and attention mechanisms to capture temporal dependencies. Tang et al. [23] employed predictive techniques to amplify the distinction between normal and anomaly events. Subsequently, they fortified the predictive frames of normal occurrences through a reconstruction approach, thereby enhancing robustness. Tsatsral et al. [24] utilized an autoregressive model to ascertain the optimal window length for subsequences, followed by the deployment of a deep autoencoder to learn the data distribution, detecting anomalies via reconstruction error. Li et al. [25] proposed an unsupervised method named InterFusion for the detection and interpretation of anomalies in multivariate time series data. This approach entails learning cross-dimensional and temporal representations of multivariate time series to compute reconstruction probabilities. Subsequently, a Markov Chain Monte Carlo technique is employed to obtain more precise anomaly reconstructions, thereby achieving effective unsupervised anomaly detection and interpretation. Gao et al. [26] developed the Time Series Memory Augmented Autoencoder (TSMAE) for anomaly detection in Internet of Things (IoT) data. By incorporating penalty and correction mechanisms into the loss function, TSMAE encourages the sparsity of address vectors. The memory module extracts normal patterns, reducing model generalization and increasing the reconstruction error for anomalous samples. Jiang et al. [27] proposed HADGAN, an unsupervised anomaly detection algorithm based on GANs, tailored for hyperspectral image anomaly detection. HADGAN learns discriminative reconstruction of the background through adversarial generation networks and autoencoders, detecting anomalies via reconstruction errors.

A time series anomaly detection method based on contrastive learning

Detecting anomalies in time series datasets without explicitly labeled timestamps poses a significant challenge. To address this issue, researchers have explored the use of contrastive learning strategies. With contrastive learning, models can identify anomaly patterns in time series data in an unsupervised manner, thereby enhancing the performance of anomaly detection. For the problem of multivariate

time series anomaly detection, Yu et al. [28] proposed the Adversarial Contrastive AutoEncoder (ACAE) framework. This framework generates positive samples through multi-scale timestamp masking and random sampling for negative samples. A discriminator is designed to perform feature decomposition, distinguishing between positive and negative compound features, and predicting the proportion of original sample features in the compound features. Through adversarial learning, the model captures the commonalities of normal time series data, obtaining robust representations. Zhou et al. [29] enhanced data in the time and frequency domains to generate two views for each time series segment and compared and learned time-dependent and local invariant features using context and instances. Kim et al. [30] designed a contrastive learning-based approach for multivariate time series anomaly detection, termed CTAD. This method leverages the principles of one-class classification, where all normal segments within the training dataset are designated as positive samples, while anomalous segments generated through negative data augmentation are classified as negative samples. Miao et al. [31] introduced an anomaly detection method based on generative adversarial networks (GANs). The training set is expanded through data augmentation using masks based on geometric distributions. Within the GAN framework, a Transformer-based autoencoder is trained to learn the latent distribution of normal data. A contrastive loss is incorporated into the discriminator to enhance generalization capabilities and adjust the network. Jin et al. [32] introduced a multi-scale contrastive learning framework named ANEMONE, which incorporates two graph neural network-based contrastive networks. These networks respectively learn patch-level contrasts between node pairs and context-level contrasts between nodes and their networks. The framework utilizes a contrastive learning module to enforce consistency in node representations across different scales. Liu et al. [33] sampled pairs of contrasting instances between target nodes and local subgraphs, employing a Graph Neural Network (GNN) model to learn the representations of each instance pair. Zheng et al. [34] employed multi-view contrastive learning to compare the representations of target nodes with surrounding subgraphs in both embedding and structural spaces, thereby detecting structural anomalies and mixed anomalies of structural attributes.

Cloud system performance anomaly detection method

In the domain of cloud computing, the detection of performance anomalies in cloud systems is critical for maintaining system stability and ensuring high-quality service, a need that has intensified with the widespread adoption of cloud computing technologies. Chen et al. [35] introduced ADSketch,

an interpretable and adaptive performance anomaly detection method based on pattern sketching, designed for online service systems. This approach detects anomalies by extracting normal and anomalous patterns from metric time series and comparing them against established patterns. By associating extracted patterns with specific performance issues, it facilitates the rapid identification of performance problems. Tania et al. [36] identify anomalies by comparing Dirichlet process Gaussian mixture models with different time lags and calculating distance scores between models. Anomalies in cloud data centers are indicated when there is a significant discrepancy between models with short-time lags while those with long-time lags remain relatively consistent. He et al. [21] proposed a topological multivariate time series anomaly detection method for cloud computing systems. This method initially integrates system topology information and employs a graph neural network to capture spatial dependencies. Subsequently, a variational autoencoder (VAE) is utilized for anomaly detection, addressing data contamination during the training process. Additionally, they have developed a threshold selection strategy based on data distribution. Shaifu et al. [37] employed a multi-classifier optimized with triplet loss to minimize the distances between similar bottleneck patterns in cloud servers while maximizing the distances between dissimilar patterns. To prevent catastrophic forgetting in the model, they have introduced a teacher-student network strategy based on knowledge distillation. For time-critical Internet of Things (IoT) applications, Lyu et al. [38] leveraged the processing capabilities of fog computing platforms to send terminal node data to the nearest fog node for analysis and classification, thereby reducing communication latency and alleviating constraints on data processing latency and memory bottlenecks. Subsequently, they enhance the accuracy of anomaly detection through information sharing between fog and cloud layers. Mostafa et al. [39] proposed a method for detecting operational anomalies in cloud applications using log and metric correlation analysis. Initially, regression analysis is employed to select metrics closely related to operational steps. The results of the regression models are then used to derive assertion specifications for runtime validation of operation execution. Baek et al. [40] clustered cloud network datasets to generate reference labels, upon which they establish a supervised learning model. Finally, they test individual data points using the established model to determine their anomaly status.

Comprehensive analysis

In cloud environments, the diversity of tasks handled by different servers can lead to significant variations among normal data samples, while anomalous events may manifest in multiple patterns. Consequently, training strategies that solely focus on reducing reconstruction error may fall short

in extracting deep semantic information about the resource states of cloud systems. Contrastive learning, which obtains effective data representations by analyzing the differences between samples, can enhance the discriminative power of models. However, existing anomaly detection methods based on contrastive learning have not fully utilized the topological structure information of cloud systems. This structural information is crucial for a profound understanding of the system's operational state. To address this, we propose a performance anomaly detection model for cloud systems that integrates Graph Neural Networks (GNNs), contrastive learning, and reconstruction methods. This model can comprehensively utilize temporal and spatial information within cloud systems, providing an effective solution for the diagnosis of performance anomalies in cloud systems.

Methodologies

In this section, we outline the anomaly detection problem. Subsequently, we offer a comprehensive description of the GCAD model's architecture and the implementation strategies for its core functional modules.

Problem description

In the context of cloud systems, which consist of numerous servers, each performing unique roles and duties, complex interdependencies emerge among them. To effectively detect performance anomalies within cloud systems, it is crucial to consider two categories of information. The first concerns the topological structure of the cloud server clusters, encompassing the relationships between servers, such as the communication between management nodes and worker nodes. The second pertains to the runtime resource status of the servers, which includes metrics such as CPU usage, memory utilization, and disk I/O rates at any given moment. Consider each server as a node, with each node associated with a set of multivariate time series that reflect the resource variations during server operation. The interconnections between servers are represented as edges between nodes, thus forming a time series that incorporates a graph structure, as illustrated in Fig. 2.

In Fig. 2, "Master" denotes the management server within the cloud system, whereas "Node" refers to the worker server. The notation $c^{1,1}$ represents the CPU utilization of the Master server at the 1st moment, and $m^{\tau,2}$ signifies the memory utilization of Node1 at the τ -th moment.

Let $\mathcal{S} \in \mathbb{R}^{L \times D}$ denote the time series data recorded during the operation of a cloud system, where L represents the length of the sequence, and D indicates the dimensionality of the sequence. The dimension D is defined as $\mathcal{C} \times \Upsilon$, with Υ being the total number of physical servers deployed within

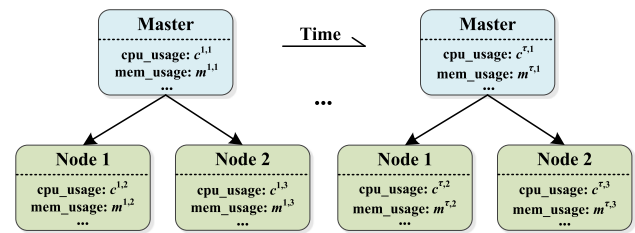


Fig. 2 Graph structure time series

the cloud system, and $\mathcal{C}$ denoting the number of performance metrics for each server. Consequently, $\mathcal{S}$ can be expressed as $\mathcal{S} = \{s_1, s_2, \dots, s_L\}$, where $s_l \in \mathbb{R}^D$ represents the observation vector of $\mathcal{S}$ at time step l . The inter-server connection relationships can be represented by the edge matrix E . In the realm of cloud system anomaly detection, the first step involves calculating anomaly scores by monitoring the observation vectors at each time step within the sequence. Subsequently, an appropriate threshold is selected to determine whether each time step is anomalous.

Due to the necessity for cloud systems to maintain continuous operation over extended periods, the volume of data accumulated from their monitoring is consequently substantial. Utilizing the entire time series as input for a model would consume considerable computational resources, and the long-distance dependencies within the sequence could potentially lead to non-convergence of the model. To reduce training costs while preserving the temporal correlations between adjacent time steps, we employ a sliding window approach with a length of T and a stride of q , to extract contiguous data segments for model input. Let $X = [x_1, x_2, \dots, x_T]$ represent a sample obtained by the sliding window, where $X \in \mathbb{R}^{T \times D}$, and T denotes the length of X . $x_t \in \mathbb{R}^D$ signifies the t -th observation vector within X , with $x_t = [a_1^t, a_2^t, \dots, a_D^t]^T$, and a_d^t represents the d -th feature value in vector x_t . By using the samples obtained from the sliding window as model input, the task of anomaly detection in cloud systems can be formalized as follows:

$$(y_1, y_2, \dots, y_T) = \text{Anomaly}(x_1, x_2, \dots, x_T; E), \quad (1)$$

where $\text{Anomaly}(\cdot)$ denotes the anomaly detection model, and $y_t \in \{0, 1\}$ represents the state of the observation vector x_t . A y_t value of 0 indicates that x_t is in a normal state, whereas a y_t value of 1 signifies an anomalous state.

Anomaly detection workflow

The workflow for anomaly detection in cloud systems is depicted in Fig. 3. Initially, monitoring tools are employed to collect performance data generated during the operation of the cloud system, while the system's topological structure information is represented as an array. During the model

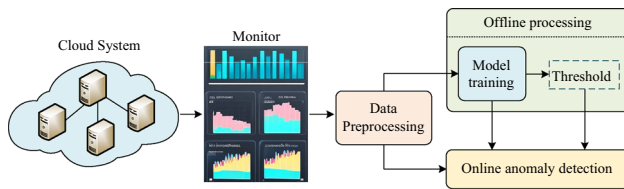


Fig. 3 Cloud system anomaly detection workflow

training phase, offline batch processing is employed to process and analyze the performance and structural data. After sufficiently training the model, a threshold for anomaly scores is determined to differentiate between normal and anomaly system behavior. During the online anomaly detection stage, real-time monitoring data from the cloud system is preprocessed and then fed into the trained model. The model continuously computes anomaly scores for the newly observed performance data. If the anomaly score for a particular observation exceeds the predefined threshold, it is determined as an anomaly.

Anomaly detection model

Overall architecture

The GCAD model we propose encompasses a data augmentation module, a spatiotemporal feature fusion module, a contrastive learning module, and a reconstruction module. The architecture of the model is depicted in Fig. 4.

According to Fig. 4, the information processing process of the GCAD model is as follows. Initially, we apply a sliding window technique to the raw data to extract data segments of a consistent length. Subsequently, these data segments are augmented in both the time and frequency domains. In this process, samples collected from the same sliding window are treated as positive pairs after processing, while samples from different time windows are considered negative examples. Following this, the GraphGRU module is employed to fuse the features of the temporal data and the topological structure, thereby acquiring spatiotemporal embedding representations. A linear attention mechanism with shared weights serves as the encoder to extract features from the embedding representations, resulting in a global embedding vector. Concurrently, projection heads are introduced to enhance the model's non-linear representational capacity, thereby optimizing the embedding vectors. During the model training phase, we employed the contrastive loss function InfoNCE to optimize the training effectiveness by maximizing the similarity between positive pairs and minimizing the similarity between negative examples. Moreover, a decoder reconstructs the original sequences using the encoder's output representations and updates the network weights based on the reconstruction error.

Data augmentation module

Data augmentation is a technique that effectively reduces the risk of overfitting in models by increasing the quantity and quality of training data. In the field of computer vision (CV), common data augmentation methods include rotations, scaling, and color transformations. However, these methods do not apply to performance data in cloud systems, as they might alter the inherent high-level semantic features within the time series data. We propose a novel data augmentation strategy to ensure efficient extraction of feature information from multivariate time series. This strategy expands the monitoring data of different metrics in the time and frequency domain, thereby thoroughly exploring and utilizing the representational information of time series data.

In the time domain, we employ a stochastic masking technique [31] grounded in the geometric distribution to manipulate the raw data. We commence by constructing a Markov chain encompassing two states: "Mask" and "Normal." This chain is instrumental in generating a binary noise mask matrix, denoted as $\mathcal{K}$. The state transition probability matrix is formulated as follows:

$$F = \begin{bmatrix} f_{00} & f_{01} \\ f_{10} & f_{11} \end{bmatrix} = \begin{bmatrix} f_{\alpha:\alpha} & f_{\alpha:\beta} \\ f_{\beta:\alpha} & f_{\beta:\beta} \end{bmatrix}, \quad (2)$$

where $f_{\alpha:\beta}$ denotes the probability of transitioning from the "Mask" state to the "Normal" state. Given a masking ratio p and the average length of the mask segment η_α , we derive the transition probabilities for the "Mask" state and the "Normal" state from the geometric distribution can be represented by $f_{\alpha:\beta} = \frac{1}{\eta_\alpha}$ and $f_{\beta:\alpha} = f_{\alpha:\beta} * \frac{p}{1-p}$, respectively. The complete transition probability matrix can be formulated as:

$$\begin{aligned} F(b_{t+1} = 0 | b_t = 0) &= f_{\alpha:\alpha} = 1 - \frac{1}{\eta_\alpha}; \\ F(b_{t+1} = 1 | b_t = 0) &= f_{\alpha:\beta} = \frac{1}{\eta_\alpha}; \\ F(b_{t+1} = 0 | b_t = 1) &= f_{\beta:\alpha} = \frac{1}{\eta_\alpha} * \frac{p}{1-p}; \\ F(b_{t+1} = 1 | b_t = 1) &= f_{\beta:\beta} = 1 - \frac{1}{\eta_\alpha} * \frac{p}{1-p}. \end{aligned} \quad (3)$$

The application of mask operations to the performance data $X \in \mathbb{R}^{T \times D}$ of a cloud system results in augmented data, as shown in Eq. (4).

$$\hat{X} = \mathcal{K} \odot X, \quad (4)$$

where $\mathcal{K} \in \{0, 1\}^{T \times D}$ represents a binary noise mask matrix adhering to a geometric distribution, while $\hat{X}$ denotes the augmented data in the temporal domain.

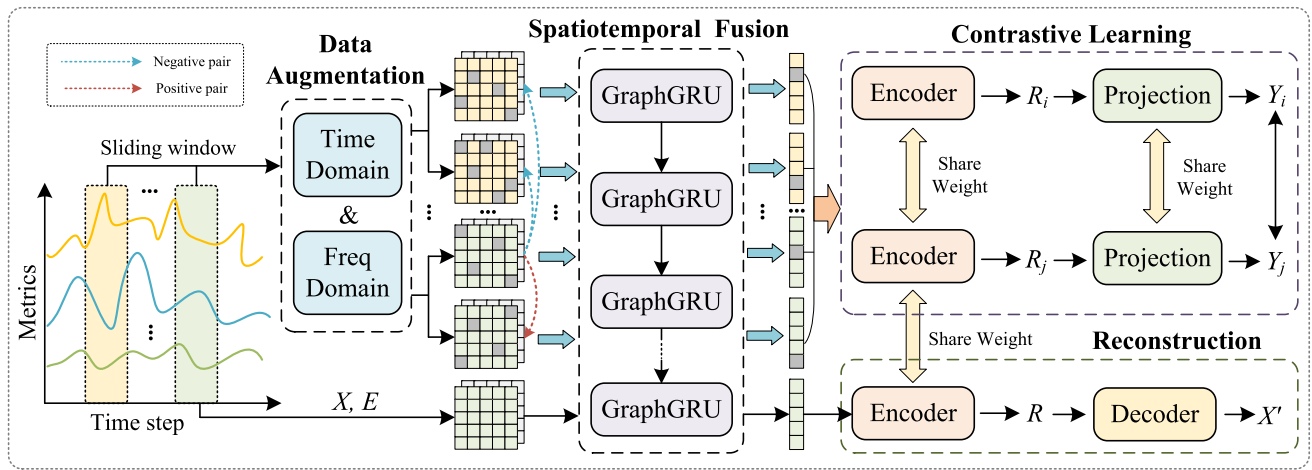


Fig. 4 The overall architecture of GCAD

In the frequency domain, a two-dimensional discrete Fourier transform is applied to the performance data $X = [x_1, x_2, \dots, x_T]$ of the cloud system to obtain the spectrum $FS(u, v)$.

$$FS(u, v) = \sum_{t=0}^{T-1} \sum_{d=0}^{D-1} a_d^t \exp\left(-\xi 2\pi \left(\frac{ut}{T} + \frac{vd}{D}\right)\right) \quad (5)$$

$$= A(u, v) \exp(\xi \phi(u, v)),$$

where a_d^t denotes the d -th feature within x_t , with $u = 0, 1, \dots, T-1$ and $v = 0, 1, \dots, D-1$ serving as coordinates in the frequency domain. T and D represent the window size and dimensionality of the sequence X , respectively, while ξ is the imaginary unit. The spectrum $F(u, v)$ can be represented as a combination of the amplitude spectrum $A(u, v)$ and the phase spectrum $\phi(u, v)$.

Gaussian noise $\mathcal{N}(\mu, \sigma^2)$, where μ denotes the mean and σ^2 represents the variance, is added to both the amplitude spectrum $A(u, v)$ and the phase spectrum $\phi(u, v)$. The mean is set to zero to avoid altering the direct current (DC) component of the original signal. Subsequently, an inverse discrete Fourier transform (IDFT) is performed to convert the frequency-domain data back to the time domain, with the real part of the resulting data being considered as the augmented data. The formulation is as follows:

$$\tilde{X} = \text{Re} \left[\frac{1}{TD} \sum_{u=0}^{T-1} \sum_{v=0}^{D-1} FS(u, v) \exp\left(\xi 2\pi \left(\frac{ut}{T} + \frac{vd}{D}\right)\right) \right], \quad (6)$$

where $t = 0, 1, \dots, T-1$ and $d = 0, 1, \dots, D-1$ denote the time steps and dimensions of the augmented data, respectively, while $\text{Re}(\cdot)$ is the operator for extracting the real part of a complex number.

Due to the zero mean attribute of Gaussian noise, it will not have a sustained impact on the total energy level and mean of the signal. The additive property of Gaussian noise ensures that the morphology and characteristics of the original signal remain essentially unaltered. With appropriate control of the noise level, the preservation of the signal's information integrity is feasible. In addition, the Gaussian noise incorporated into the signal shows up as random variations, not systematic errors, so it does not alter the statistical properties of the signal.

GraphGRU module

To model graph-structured time series data within cloud systems, we design the GraphGRU model, which integrates GNNs with GRUs. Specifically, we replace the conventional fully connected layers in the GRU cells with GNN layers to accommodate the characteristics of graph-structured data. The GRU, equipped with gating mechanisms, effectively captures long-term dependencies within time series data. Compared to LSTM networks, the GRU has a more streamlined structure and fewer parameters, typically resulting in faster training speeds. Figure 5 illustrates the internal architecture of the GraphGRU model.

In the GraphGRU module, all states and gates are represented in a graph-based format. This representation incorporates the features of each node as well as the topological structure represented by the set of edges E . When the GraphGRU receives an input vector x_t at time t , the reset gate r_t determines the extent to which the previous state is ignored. If the reset gate is closed (nearing 0), the previous hidden state is disregarded; if open (nearing 1), the previous hidden state is utilized to generate a new candidate state. The update gate z_t decides the degree to which the old hidden state is updated with the new candidate's hidden state. The candidate hidden

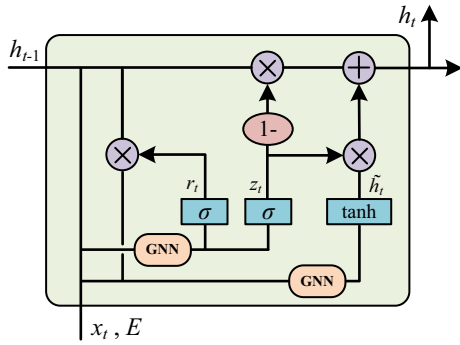


Fig. 5 The structure of GraphGRU

state $\tilde{h}_t$ integrates the preceding hidden state with the current input information, enabling the capturing of long-term dependencies within the sequence. The GraphGRU can be formulated as follows:

$$\begin{aligned} r_t &= \sigma(W_r * \mathcal{G}([h_{t-1}, x_t], E) + b_r); \\ z_t &= \sigma(W_z * \mathcal{G}([h_{t-1}, x_t], E) + b_z); \\ \tilde{h}_t &= \tanh(W_h * \mathcal{G}([r_t \odot h_{t-1}, x_t], E) + b_h); \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \end{aligned} \quad (7)$$

where $\mathcal{G}$ denotes the graph convolution operation, $\odot$ represents the element-wise product, and $\sigma(\cdot)$ is the Sigmoid activation function.

Inspired by advanced GNNs [41], we have incorporated an attention mechanism into the graph convolutional process to enhance the representation of inter-node correlations. The output of the $(\ell+1)$ th layer in graph convolution can be formulated as:

$$H^{\ell+1} = \sigma(\Xi^\ell H^\ell W^\ell), \quad (8)$$

where W^ℓ is a trainable weight matrix, $\sigma(\cdot)$ is the activation function, $H^\ell \in \mathbb{R}^{\Upsilon \times \mathcal{T}}$ is the representation matrix obtained at the ℓ th layer, Υ is the number of servers, i.e., the number of nodes in the graph, and $\mathcal{T}$ is the embedding dimension of the nodes. Ξ^ℓ represents the attention coefficient matrix at the ℓ th layer, which can be calculated as:

$$\Xi_{ij}^\ell = \frac{\exp(\mathcal{J}_{ij}^\ell)}{\sum_{t \in \Pi_i} \exp(\mathcal{J}_{it}^\ell)}, \quad (9)$$

where Π_i represents the set of all neighborhoods of node i , $\mathcal{J}^\ell$ is the affiliation matrix at the ℓ th layer, which can be determined as follows:

$$\mathcal{J}^\ell = \text{sigmoid}(\tilde{E} \odot \lambda_1^\ell H^\ell W^\ell + \tilde{E} \odot \lambda_2^\ell H^\ell W^\ell), \quad (10)$$

In Eq. (10), $\tilde{E} = \mathcal{D}^{-\frac{1}{2}}(E + I_\Upsilon)\mathcal{D}^{-\frac{1}{2}}$ is the normalized adjacency matrix, I_Υ is the identity matrix, $\mathcal{D}_{ii} = \sum_j (E + I_\Upsilon)_{ij}$.

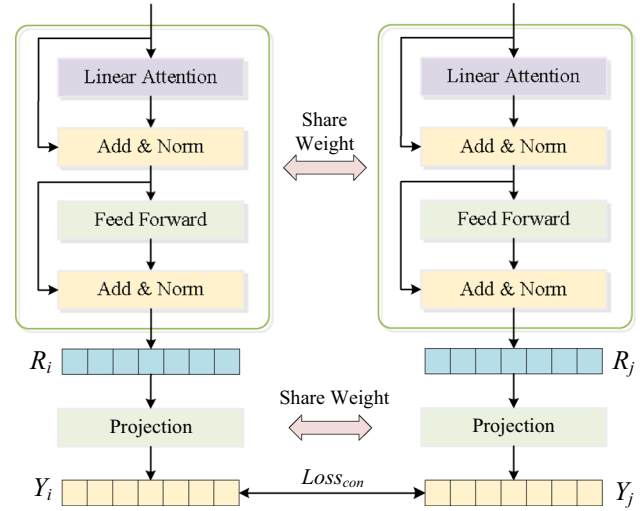


Fig. 6 Inner structure of contrastive learning module

$\lambda_1^\ell, \lambda_2^\ell \in \mathbb{R}^{1 \times d}$ are trainable parameters for the node and its neighbors, respectively.

The GraphGRU effectively controls the flow of information through its gating mechanisms, enabling the simultaneous capture of both long-term and short-term dependencies within time series data. Moreover, the GNN unit further enhances the model's capability to process graph-structured information. By aggregating feature information from neighboring nodes, the GNN takes into account the topological relationships between nodes, thereby generating representations that integrate spatiotemporal features.

Contrastive learning

The objective of contrastive learning is to obtain a feature extractor that transforms inputs into representation vectors that preserve the inherent structure and feature information. This architecture comprises an encoder and a projection layer, with the InfoNCE loss utilized to optimize the network weights. In our approach, we employ an encoder that integrates a linear attention mechanism [42] with shared weights alongside a feedforward network. This setup is designed to capture global features from the spatiotemporal representations produced by the GraphGRU. The contrastive learning module is illustrated in Fig. 6.

Unlike the self-attention mechanism, linear attention decouples the Softmax operation through appropriate approximations and alters the order of computations by first calculating $K^T V$, thereby reducing the computational complexity to $O(NC^2)$. Since the channel dimension C is typically much smaller than the length of the input N , this approach can significantly reduce the overall computational cost. The structure of the linear attention mechanism is depicted on the right side of Fig. 7.

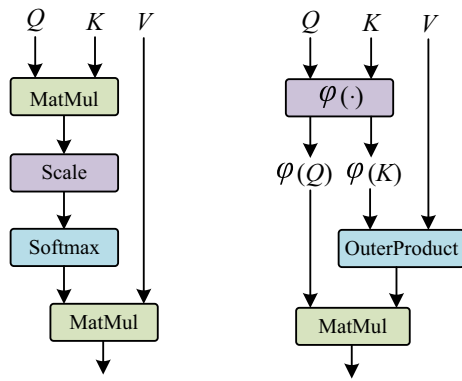


Fig. 7 Self-attention versus linear attention

In the self-attention module, the output representation $h \in \mathbb{R}^{N \times C}$ from the GraphGRU module is linearly transformed into vectors for Query, Key, and Value. This transformation can be expressed as $Q = hW_Q, K = hW_K, V = hW_V$. Consequently, the computation formula for self-attention is:

$$R_i = \frac{\sum_{n=1}^N \text{Sim}(Q_i, K_n) V_n}{\sum_{n=1}^N \text{Sim}(Q_i, K_n)}, \quad (11)$$

where W_Q, W_K and $W_V \in \mathbb{R}^{C \times C}$ denote projection matrices, while $\text{Sim}(\cdot)$ represents the similarity function. The self-attention mechanism employs the Softmax function to compute attention weights, with the similarity measure given by $\text{Sim}(Q, K) = \exp(QK^T / \sqrt{C})$. The attention map is obtained by calculating the similarity between all pairs of Q and K , incurring a computational complexity of $O(N^2)$.

In the linear attention mechanism, a kernel function is utilized as an approximation of the original similarity function. Consequently, the similarity between vectors Q and K can be represented as:

$$\text{Sim}(Q, K) = \varphi(Q)\varphi(K)^T, \quad (12)$$

where $\varphi(\cdot)$ denotes a kernel function capable of capturing feature representations. Consequently, the self-attention mechanism can be transformed as follows:

$$R_i = \frac{\sum_{n=1}^N \varphi(Q_i)\varphi(K_n)^T V_n}{\sum_{n=1}^N \varphi(Q_i)\varphi(K_n)^T}. \quad (13)$$

The computational complexity of Eq. (13) is $O(N^2C)$. By leveraging the associative property of matrix multiplication, we can alter the sequence of computations from $(QK^T)V$ to $Q(K^TV)$, thereby providing an equivalent representation of

Eq. (13):

$$R_i = \frac{\varphi(Q_i) \left(\sum_{n=1}^N \varphi(K_n)^T V_n \right)}{\varphi(Q_i) \left(\sum_{n=1}^N \varphi(K_n)^T \right)}. \quad (14)$$

The computational complexity of Eq. (14) is $O(NC^2)$.

In the process of contrastive learning, for a batch of M input samples, data augmentation generates $2M$ training samples. Specifically, each original sample's two augmented views are defined as a positive pair, while the remaining $(2M - 2)$ samples are considered negative examples. After processing the inputs with a linear attention mechanism, an output embedding vector R is produced. This embedding vector is then fed into a projection layer composed of feedforward neural networks to further extract features and introduce non-linearity, resulting in a representation Y . The computation is formalized as follows:

$$Y = FFN(R) = \max(0, R \cdot W_1 + b_1)W_2 + b_2, \quad (15)$$

where W_1, W_2, b_1 and b_2 represent the weights and biases of the feedforward neural network, respectively.

The representations produced by the feedforward network are employed for self-supervised contrastive learning. For each positive sample pair, denoted as X_i and X_j , the corresponding output feature representations are Y_i and Y_j . Consequently, the loss function can be expressed as follows:

$$\begin{aligned} \mathcal{L}_{con}^{(i,j)} &= -\log \frac{\exp(\text{CosSim}(Y_i, Y_j)/\omega)}{\sum_{k=1}^{2M} \mathbb{1}_{k \neq i} \exp(\text{CosSim}(Y_i, Y_k)/\omega)}; \\ \mathcal{L}_{con} &= \frac{1}{M} \sum_{i=1}^M \mathcal{L}_{con}^{(i,j)}, \end{aligned} \quad (16)$$

where $\text{CosSim}(\cdot)$ denotes the cosine similarity. The indicator function $\mathbb{1}_{k \neq i} \in [0, 1]$ takes the value of 1 when $k \neq i$, and 0 otherwise. The variable M represents the batch size. The loss is further modulated by an additional temperature parameter ω . This loss function is designed to maximize the similarity between the feature representations Y_i and Y_j of positive pairs, while concurrently minimizing the similarity between the feature representations Y_i and Y_k of negative pairs.

Reconstruction module

An encoder trained with contrastive learning can efficiently extract global trends and high-level semantic features from spatiotemporal data. Subsequently, we designed a two-layer MLP as the decoder to facilitate the reconstruction of the

original sequence. The reconstruction process is as follows:

$$\begin{aligned} H_g &= \sigma(W_{g1} \cdot R + b_{g1}); \\ X' &= \sigma(W_{g2} \cdot H_g + b_{g2}), \end{aligned} \quad (17)$$

where W_{g1} and W_{g2} represent the weight matrices for the first and second layers, respectively, while b_{g1} and b_{g2} denote the bias vectors. The symbol σ denotes the Sigmoid activation function, and $X' \in \mathbb{R}^{T \times D}$ represents the reconstructed sample of X .

The loss of reconstruction calculated by Mean Squared Error (MSE) can be expressed as:

$$\mathcal{L}_{oss_{rec}} = \frac{1}{\mathcal{M}T} \sum_{i=1}^{\mathcal{M}} \sum_{t=1}^T (x'_{it} - x_{it})^2, \quad (18)$$

where $\mathcal{M}$ denotes the batch size, T represents the size of the sliding window, and x'_{it} and x_{it} correspond to the reconstructed vector and the original vector of the t -th time step for the i -th sample within a batch, respectively.

Joint loss optimization

The performance data of the cloud system contains noise and a few anomalies. To reduce the risk of overfitting the model to these suboptimal data, we introduced a Maximum Mean Discrepancy (MMD) regularization term in the low-dimensional feature space [22]. The core concept of MMD is to compare the means of samples from two distributions in the Reproducing Kernel Hilbert Space (RKHS). An MMD value of zero implies congruent distributions, while a positive value indicates discrepancies. By minimizing the MMD, we aim to elevate the approximation between the distribution of the low-dimensional representation and the target distribution, thereby mitigating the impact of noise and anomalies in the training data. We choose the Gaussian distribution $\tilde{\rho}$ as the target distribution. For the low-dimensional representation $h = \{h^{(1)}, \dots, h^{(N)}\} \in \mathbb{R}^{N \times C}$ yielded by the GraphGRU module, the MMD calculation formula is presented as follows:

$$\mathcal{L}_{oss_M}(\tilde{h}, \tilde{\rho}) = \left\| \frac{1}{N} \sum_{i=1}^N \psi(h^{(i)}) - \frac{1}{N} \sum_{i=1}^N \psi(\rho^{(i)}) \right\|_{\mathcal{H}}^2, \quad (19)$$

where $\tilde{h}$ represents the distribution of the low-dimensional representation $h^{(i)}$, $\tilde{\rho}$ denotes the target distribution, and $\rho^{(i)} \sim \tilde{\rho}$. MMD is defined by the feature mapping $\psi: X \rightarrow \mathcal{H}$, where $\mathcal{H}$ is a RKHS. In practical calculations, MMD is

generally approximated by the following empirical formula:

$$\begin{aligned} \mathcal{L}_{oss_M}(\tilde{h}, \tilde{\rho}) &= \left\| \frac{1}{N^2} \sum_{i \neq j} k(h^{(i)}, h^{(j)}) \right. \\ &\quad \left. + \frac{1}{N^2} \sum_{i \neq j} k(\rho^{(i)}, \rho^{(j)}) - \frac{2}{N^2} \sum_{i,j} k(h^{(i)}, \rho^{(j)}) \right\|_{\mathcal{H}}, \end{aligned} \quad (20)$$

where the kernel function is defined as $k(u, v) = \exp\left(-\frac{\|u-v\|^2}{2\sigma^2}\right)$.

Consequently, the comprehensive training loss function is represented by the aggregate of the reconstruction loss, contrastive loss, and regularization term:

$$\mathcal{L} = \mathcal{L}_{oss_{rec}} + \mu_1 \mathcal{L}_{oss_{con}} + \mu_2 \mathcal{L}_{oss_M}, \quad (21)$$

where μ_1 and μ_2 are the hyperparameters controlling the relative importance of the losses. The complete training procedure is detailed in Algorithm 1.

Algorithm 1 Anomaly detection

Input: Pre-processed dataset X , initialized GraphGRU $G(\cdot)$, initialized Encoder $E(\cdot)$, initialized Decoder $D(\cdot)$

Output: Trained GraphGRU $G(\cdot)$, Encoder $E(\cdot)$, and Decoder $D(\cdot)$

1: **while** convergence **do**

2: Obtain positive and negative samples pairs

3: Input all samples into $G(\cdot)$ to obtain the spatiotemporal fusion representation

4: **Compute MMD:**

$$\mathcal{L}_{oss_M}(\tilde{h}, \tilde{\rho}) = \left\| \frac{1}{N} \sum_{i=1}^N \psi(h^{(i)}) - \frac{1}{N} \sum_{i=1}^N \psi(\rho^{(i)}) \right\|_{\mathcal{H}}^2$$

5: Input a pair of representations into Eq. 14 to obtain the feature vectors

6: Input feature vectors into Eq. 15

7: **Compute contrastive loss:**

$$\mathcal{L}_{oss_{con}} = \frac{1}{\mathcal{M}} \sum_{i=1}^{\mathcal{M}} \left[-\log \frac{\exp(\text{sim}(Y_i, Y_j)/\tau)}{\sum_{k=1}^{2\mathcal{M}} \mathbf{1}_{k \neq i} \exp(\text{sim}(Y_i, Y_k)/\tau)} \right]$$

8: Optimize $G(\cdot)$ and $E(\cdot)$ to minimize $\mathcal{L}_{oss_M}$ and $\mathcal{L}_{oss_{con}}$

9: Reconstruct X using $G(\cdot)$, $E(\cdot)$, and $D(\cdot)$

10: **Compute reconstruct loss:**

$$\mathcal{L}_{oss_{rec}} = \frac{1}{\mathcal{M}T} \sum_{i=1}^{\mathcal{M}} \sum_{t=1}^T (x'_{it} - x_{it})^2$$

11: Optimize $D(\cdot)$ to minimize $\mathcal{L}_{oss_{rec}}$

12: **end while**

In the offline training phase, we optimize parameters by minimizing the contrastive loss and the reconstruction loss to capture the normal patterns of resource states in cloud systems. In the testing phase, we utilize the encoder and decoder to reconstruct the test samples. Then, we compute the anomaly score for each time step based on the reconstruction error. The formula is as follows:

$$Z_{it} = (x'_{it} - x_{it})^2, \quad (22)$$

where Z_{it} represents the anomaly score for the t -th time step of the i -th test sample.

Threshold selection

In the offline training process, we compute an anomaly score for each observation vector of the multivariate sequence. Thus, the corresponding anomaly score for a sliding window sample can be represented as the sequence $Z = \{Z_1, Z_2, \dots, Z_T\}$. A higher anomaly score indicates a larger reconstruction error. We set a threshold ε for the anomaly scores, where an observation is deemed anomalous if its anomaly score exceeds ε , and normal otherwise.

We employ a threshold selection method [21] that begins by sorting the set of anomaly scores Z in ascending order to form Z' . Given that the distribution of anomaly scores for normal data is significantly different from that of anomalous data, we define two subsets of anomaly scores, $Z'_{\varepsilon-}$ and $Z'_{\varepsilon+}$, which are separated by the threshold ε . The distance between $Z'_{\varepsilon-}$ and $Z'_{\varepsilon+}$ can be expressed as:

$$d(Z'_{\varepsilon-}, Z'_{\varepsilon+}) = \frac{\min(Z'_{\varepsilon+}) - \max(Z'_{\varepsilon-})}{\min(Z'_{\varepsilon+}) + \max(Z'_{\varepsilon-}) - 2 * \min(Z'_{\varepsilon-})}, \quad (23)$$

where $\max(Z'_{\varepsilon-})$ denotes the maximum element within $Z'_{\varepsilon-}$, and $\min(Z'_{\varepsilon+})$ represents the minimum element.

In selecting a threshold ε from Z' , our objective is to maximize the distance between the two subsets:

$$\varepsilon = \operatorname{argmax}(d(Z_{\varepsilon-}, Z_{\varepsilon+})). \quad (24)$$

Experiments

In this section, we validate the efficacy of our proposed method on one custom dataset and two publicly available datasets. To facilitate a comprehensive evaluation, we select eight state-of-the-art baseline models for comparison. Furthermore, we design ablation studies to investigate the significance and necessity of each module within our model. Lastly, we provide a detailed analysis of the performance of the GCAD through a case study.

Experimental setup

Dataset

(1) Slurm Dataset(SlurmD). To mimic a genuine cloud environment, we establish a Slurm cluster [43] comprising one master server and seven worker servers. To comprehensively monitor and ensure the stable operation of this cloud

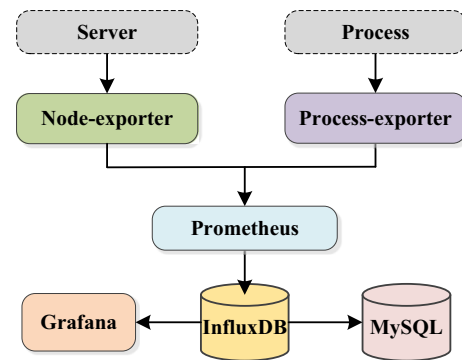


Fig. 8 Monitoring system workflow

system, we have implemented a suite of open-source monitoring tools, including Prometheus [44], Node_exporter, Process_exporter, Grafana [45], InfluxDB [46], and MySQL. In our experimental setup, Prometheus serves as the primary monitoring tool, periodically pulling metric data from Node_exporter and Process_exporter. Node_exporter is deployed on all monitored servers to collect hardware and system-level metrics, while Process_exporter is utilized for monitoring specific processes. InfluxDB, a time series database, is used to store a large volume of monitoring data. To ensure the long-term preservation of monitoring data and facilitate complex data analysis, we regularly export data from InfluxDB to a MySQL database. Grafana is employed for the visualization of monitoring data. The operational flow of the monitoring system is depicted in Fig. 8.

The objective of this study is to detect abnormal states in server performance. To achieve this, we utilized a monitoring system to collect a comprehensive set of metrics related to the server's CPU, memory, disk, and network activities. A detailed list of these metrics is provided in Table 1. To simulate both normal and abnormal workloads, we utilized stress-testing tools, ensuring that these workloads were generated at random intervals across different periods. Considering the relatively low frequency of abnormal events in real-world cloud environments, we set the probability of anomaly occurrence at 5.2%. For simulating CPU, memory, and disk-related faults, we chose StressLinux7, a versatile tool for stress testing and system debugging. To mimic network-related failures, we employed iPerf8, a network performance testing tool, which we used to simulate network congestion by occupying bandwidth with TCP and UDP packets.

(2) MBD [47] is harvested from a cluster consisting of a master server and four slave servers. Each node encompasses 26 performance metrics, including CPU system, CPU user, disk I/O wait per second, and memory activity, among others. Anomalous events encompass application failures, high CPU and memory utilization, disk I/O usage, and network latency

Table 1 Resource metrics recorded from Slurm

Metrics	Description
cpu_second_total	The total CPU usage of the server
cpu_second_iowait	The proportion of CPU time waiting for I/O
cpu_seconds_total (mode=user)	CPU usage in user mode
cpu_seconds_total (mode=system)	CPU usage in system mode
MemTotal	The total memory capacity
MemAvailable	The available memory on the server
SwapFree	The free space of the swap partition
SwapTotal	The total size of the swap partition
filesystem_avail	The available capacity of the file system
filesystem_size	The total capacity of the file system
disk_io_time	The disk I/O time
disk_read	Total bytes read from disk
disk_write	Total bytes written to disk
network_receive	The total bytes received over network
network_transmit	The total bytes transmitted over network

and packet loss. The incidence probability of such anomalous events is 6.55%.

(3) Server Machine Dataset (SMD) [48]. SMD is a 5-week data set collected by a large Internet company. It is amalgamated from data across 28 distinct machines, with each machine's data structured around 38 dimensions. These dimensions are indicative of various performance metrics of the machines. The incidence rate of anomalies within this dataset is 4.16%. In consideration of the lack of direct interrelation among the machines in the dataset, we have employed it to evaluate the performance of our model under conditions where the GraphGRU module is non-functional.

Baseline

In this work, we compare the proposed methodology with eight established anomaly detection techniques, as delineated below:

LOF [49]: This technique identifies local outliers within a dataset by quantifying the degree of isolation of each data point relative to its local neighborhood, offering a quantitative measure of outlierness.

TranAD [50]: An anomaly detection model based on the Transformer architecture that employs adversarial training to amplify reconstruction errors.

BeatGAN [51]: An unsupervised anomaly detection model for time series data that utilizes generative adversarial networks for data reconstruction and regularization, while incorporating time series warping for data augmentation.

LSTM-NDT [52]: The approach employs LSTM to learn the normal data of spacecraft, establishing a model of normal behavior. Subsequently, it utilizes a non-parametric approach

to dynamically set anomaly thresholds, thereby determining the occurrence of anomaly states.

TimeCLR [53]: A contrastive learning framework for time series representations that employs DTW for data augmentation and InceptionTime for feature extraction.

ACAE [28]: A robust multivariate time series anomaly detection framework.

TopoMAD [21]: A stochastic seq2seq model based on graph neural networks, LSTM, and VAEs for unsupervised anomaly detection in cloud systems.

CAE-AD [29]: A contrastive autoencoder for multivariate time series anomaly detection.

CGAN [31]: A framework for anomaly detection in multivariate time series data, utilizing contrastive learning and generative adversarial networks.

AMFormer [54]: An approach to unsupervised anomaly detection in time series data, employing the Transformer architecture and incorporating a masking operation mechanism and cost-sensitive learning theory.

Performance metrics

To evaluate the effectiveness of our anomaly detection approach, we utilized precision (P), recall (R), and the F1 score as our performance metrics. Precision measures the proportion of correctly identified anomaly instances among all instances classified as anomalies. It can be calculated as follows:

$$P = TP / (TP + FP). \quad (25)$$

Recall measures the proportion of anomaly instances correctly identified among all truly anomalous instances. It can

Table 2 Comparison results of different models on cluster-trace-v2020

Baseline	SlurmD			MBD			SMD		
	P	R	F1	P	R	F1	P	R	F1
LOF	0.530	0.507	0.518	0.214	0.279	0.242	0.648	0.671	0.659
TranAD	0.834	0.857	0.845	0.413	0.375	0.393	0.926	0.997	0.961
BeatGAN	0.961	0.726	0.739	0.356	0.320	0.291	0.739	0.764	0.751
RF-LSTM	0.621	0.670	0.645	0.309	0.258	0.281	0.711	0.723	0.717
TopoMAD	0.862	0.849	0.855	0.501	0.463	0.481	0.823	0.895	0.857
TimeCLR	0.709	0.683	0.696	0.325	0.282	0.302	0.721	0.703	0.712
ACAE	0.838	0.804	0.821	0.384	0.316	0.347	0.762	0.785	0.773
CAE-AD	0.841	0.818	0.829	0.407	0.354	0.379	0.927	0.949	0.938
CGAN	0.819	0.832	0.825	0.421	0.396	0.408	0.903	0.924	0.913
AMFormer	0.804	0.781	0.792	0.385	0.401	0.393	0.865	0.997	0.926
GCAD	0.915	0.893	0.904	0.512	0.498	0.505	0.931	0.947	0.939

The data items in bold indicate the optimal outcomes obtained during the experiment

be calculated as follows:

$$R = TP / (TP + FN). \quad (26)$$

The F1 score takes into account both precision and recall, providing a comprehensive performance metric for anomaly detection. It can be calculated as follows:

$$F1 = 2 * P * R / (P + R). \quad (27)$$

Implementation details

In this study, we implement the GCAD model utilizing the Pytorch framework. During the data augmentation phase, we set the masking rate at 5%. For model training, we employed the Adam optimizer with a learning rate of 0.0001, a batch size of 32, and a maximum epoch of 200 to ensure thorough convergence of the model. The Gaussian noise variance is set at 0.5. To enhance the reliability of our experimental results, we adopted a fivefold cross-validation strategy. We randomly and evenly partitioned the dataset into five non-overlapping subsets, assigning one subset as the test set, and combining the other four subsets to form the training set. We then conducted a series of five training iterations using this configuration. This approach allowed for the independent evaluation of the model's performance on each subset. We averaged the experimental results obtained from the five iterations to derive a comprehensive evaluation outcome. Regarding the hardware environment, all experiments were conducted on an NF5468M5 server equipped with three NVIDIA 2080Ti GPUs.

Overall performance

In this section, we comparatively evaluate the performance of the GCAD against the baseline on three datasets, in terms of

precision, recall, and F1-score. The outcomes are presented in Table 2.

Table 2 reveals that GCAD outperforms the baseline model with precision of 0.915, 0.512, and 0.932 in anomaly detection across three datasets. TranAD exhibits the second-best performance, suggesting that attention mechanisms with global relevance are effective in capturing anomalous patterns in sequences. The average performance of TopoMAD on the three datasets is slightly inferior to that of TranAD. However, it remains competitive due to its simultaneous consideration of the cloud system's graph structure and temporal information. LOF's relatively poor performance can be attributed to its classification of all outliers as anomalies, whereas some outliers in the sequences are normative dynamic changes during the cloud system's operation. As a prediction-based model, LSTM-NDT's performance significantly deteriorates when the data does not conform to the prediction conditions. AMFormer demonstrates the optimal recall rate on SMD, signifying its heightened sensitivity to anomalies. CAE-AD and the GCAD demonstrate suboptimal performance on the SMD dataset. This indicates that contrastive learning can aid reconstruction-based models in better extracting the features of normal patterns in sequences. In addition to the performance metrics, the datasets SlurmD and MDB also incorporate correlations between different servers. Both GCAD and TopoMAD leverage GNNs to capture the associations between the performance data of machines within a cluster, hence outperforming other models on SlurmD and MDB. However, the SMD dataset only contains performance data from independent machines. When GCAD and TopoMAD perform anomaly detection on this dataset, the GNN module cannot effectively contribute, resulting in underwhelming performance. Nonetheless, the global weighting strategy of the attention mechanism combined with the joint training of multiple loss functions allows

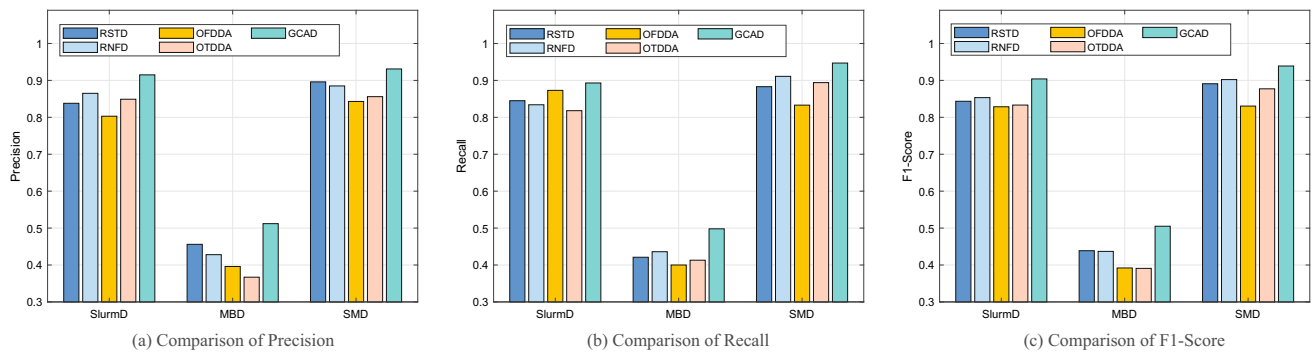


Fig. 9 Performance of different data augmentation methods

GCAD to achieve the highest precision and a near-optimal F1 score.

Ablation study

In this section, we aim to evaluate the impact of the various components of the proposed model on its performance.

Data augmentation method evaluation

We have implemented four distinct treatments on the data augmentation module, which include Random Smoothing in the Time Domain (RSTD), Random Noise in the Frequency Domain (RNFD), Only Frequency Domain Data Augmentation (OFDDA), and Only Time Domain Data Augmentation (OTDDA). Figure 9 illustrates a comparative analysis of the different data augmentation methods.

As depicted in Fig. 9, the time-frequency domain-based augmentation approach consistently obtains the best performance across all datasets. This indicates that augmenting data in both the time and frequency domains introduces more beneficial perturbations to multivariate time series data, enabling the model to generalize more effectively.

Activation function evaluation

In this section, we investigate the impact of the decoder activation function on model performance. We replace the Sigmoid activation function with Leaky ReLU, Tanh, ReLU, and ELU. Subsequently, we conduct training and testing of the model to assess the efficacy of each activation function within the decoder network. The experimental results are shown in Fig. 10.

Figure 10 illustrates that while there are variations in the performance among the activation functions, the differences are generally not substantial. Sigmoid demonstrates superior performance. The reason for this is that the nonlinear characteristics of the Sigmoid function align more closely with the distribution features of the cloud system resource data.

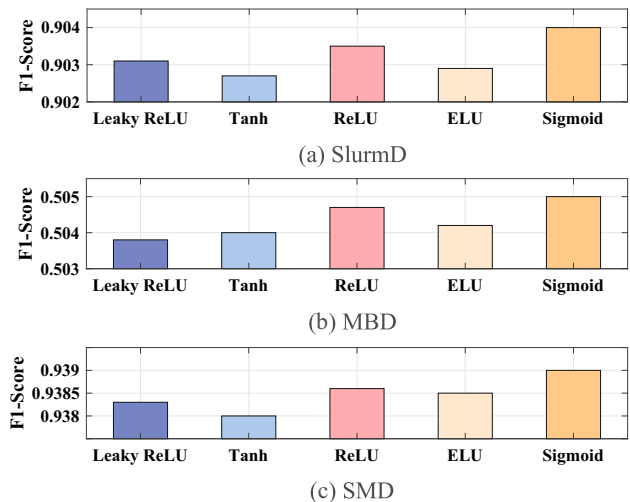


Fig. 10 Performance of different activation functions

Particularly when the data approximates a normal distribution, the Sigmoid function is more effective in capturing the underlying patterns. In contrast, although activation functions like LeakyReLU, Tanh, ReLU, and ELU each have unique nonlinear transformations, their adaptability to the specific time-series data of this study is relatively lower.

Module importance evaluation

In this section, we conducted experiments with/without crucial modules to study the effects of each module of GCAD. GCAD-1 transforms the edge matrix corresponding to the system topology into a zero matrix, disregarding the adjacency matrix. GCAD-2 replaces linear attention with self-attention. GCAD-3 substitutes the linear attention module with a multi-layer feedforward network. GCAD-4 removes the contrastive learning module. Figure 11 presents a comparison between GCAD and its four variants.

As shown in Fig. 11, GCAD achieves the best performance across four metrics. GCAD-1, which ignores the topological structure between servers, is essentially devoid of GNN

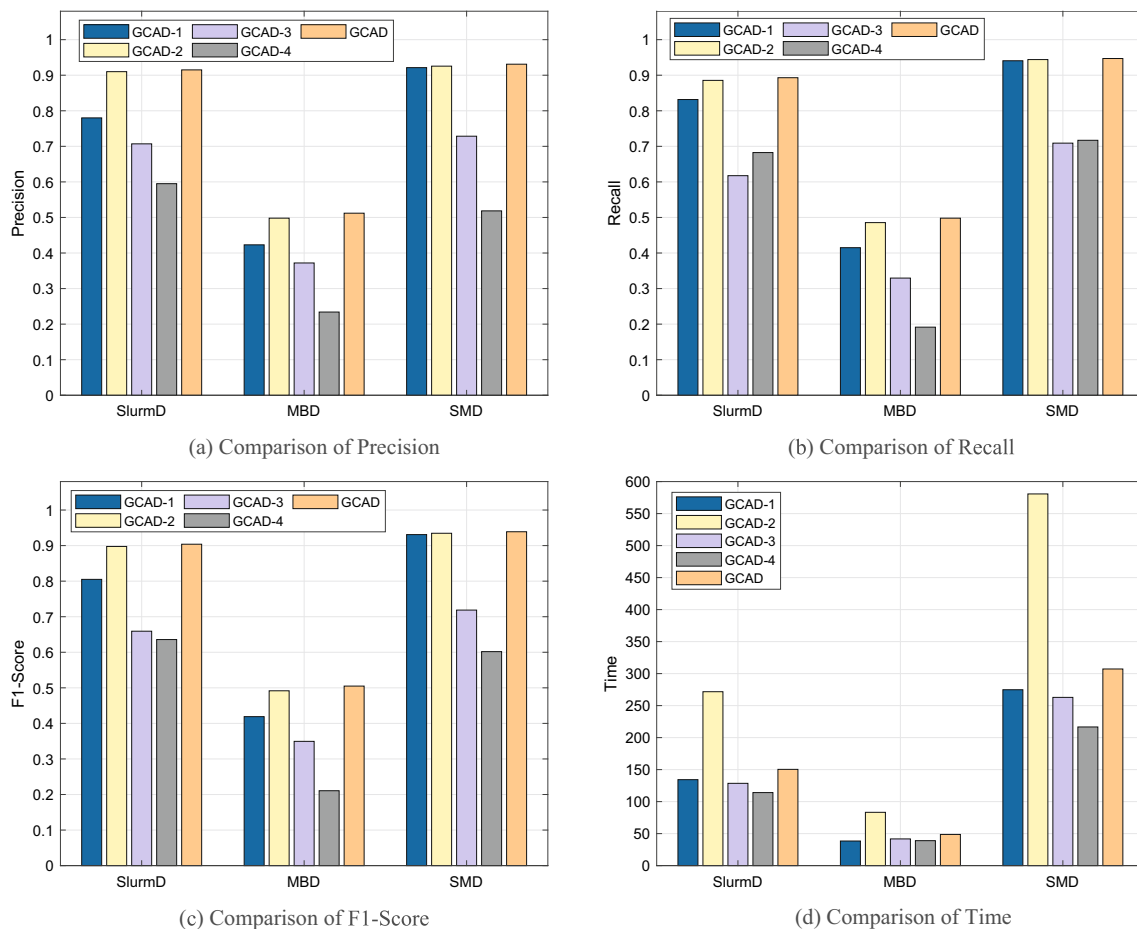


Fig. 11 Performance of GCAD and its variants

functionality, leading to a significant performance drop in the SlurmD and MBD datasets. Compared to GCAD, the F1 scores decreased by 10.94% and 17.02% respectively. However, for the SMD dataset, which only includes metrics from individual servers, the change in model performance is relatively minor.

The performance metrics of GCAD-2 on three distinct datasets reveal a decrease in average precision by 23.94%, recall by 29.94%, and F1 score by 27.11%. This result indicates that the global weighting strategy of linear attention enhances the model's information processing capability. However, replacing this with a feedforward neural network impaired the model's ability to handle long-distance dependencies, resulting in decreased performance.

The GCAD-3 model employs an encoder–decoder framework to reconstruct the original sequence and does not integrate a contrastive learning mechanism. This strategy constrains the model's ability to adequately learn the features of normal samples, thus making it difficult to obtain robust representations. The experimental findings indicate a decline in average precision, recall, and F1 score by 35.62%, 29.18%, and 33.02% across three datasets.

Due to the inherent capability of both self-attention and linear attention mechanisms to capture global interdependencies, the performance of GCAD-4 and GCAD is not significantly different. However, the quadratic complexity of self-attention substantially prolongs the training time. Compared to GCAD, the training duration of GCAD-4 on three different datasets increased by 80.67%, 70.92%, and 89.02%. These findings indicate that removing or altering critical components of GCAD can lead to a notable decline in performance or efficiency.

Parameter analysis

In this section, we delve into an in-depth exploration of the potential impact of the sliding window size and the dimensionality of the embedding vector R within the contrastive learning framework on the performance of the GCAD model. The selection of the sliding window size is inherently linked to the model's ability to capture temporal information from time series data. Meanwhile, the embedding vector R , as a crucial parameter in the contrastive learning process, plays a decisive role in the feature representations that are learned.

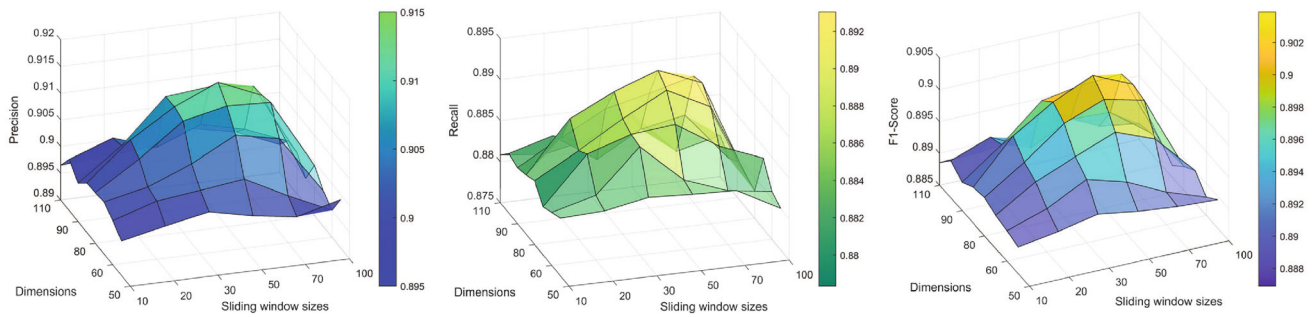


Fig. 12 Parameter analysis on SlurmD

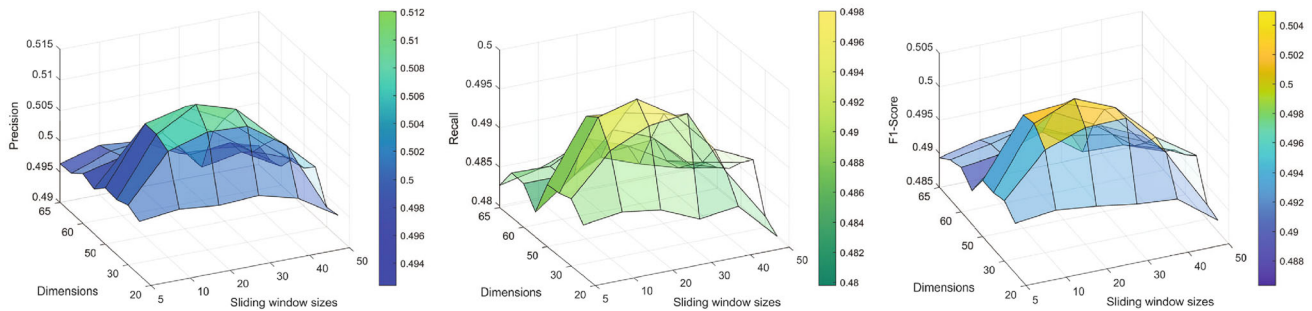


Fig. 13 Parameter analysis on MBD

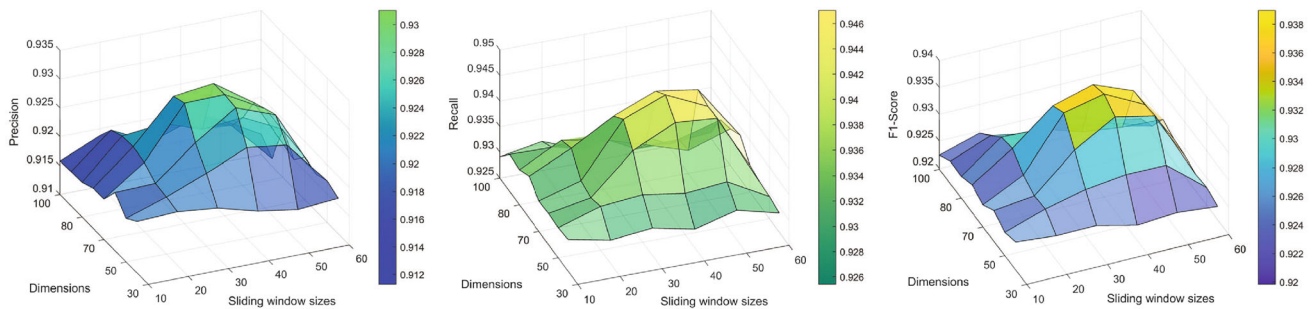


Fig. 14 Parameter analysis on SMD

Additionally, the dimensionality of the embedding vector R directly affects the model's reconstruction capabilities. The results of the parameter analysis on the three datasets are presented in Figs. 12, 13 and 14.

Figures 12, 13 and 14 illustrate the impact of varying window sizes and embedding vector dimensions on the precision, recall, and F1 score of GCAD. On the SlurmD dataset, the optimal window size and embedding dimension for GCAD are 50 and 110, respectively. For the MBD dataset, the best parameters are 20 and 50, while on the SMD dataset, they are 40 and 80.

Regarding the influence of the sliding window, when the window size is below the optimal setting, the F1-score of GCAD decreases. This is due to the limited information contained in the input sequences, which hinders the model's ability to capture complete temporal features. Moreover, if the window length is too small, the true sequence information

may be obscured by the augmented data, leading to the loss of relevant features. On the other hand, when the window size is too large, the increased number of anomaly points within the input sequences leads to a significant rise in reconstruction error, which in turn affects the accuracy of anomaly score calculation.

As for the dimensionality of the embedding vector R , when it is too small, GCAD fails to adequately capture the intrinsic structure and features of the data, thus limiting the model's representational capacity. As the dimension of R increases, GCAD can learn a finer-grained feature representation, thereby enhancing model performance until the optimal setting is reached. However, when the dimension of R exceeds the optimal setting, the heightened representational complexity allows the model to fit anomalous points too closely, which affects the reconstruction error.

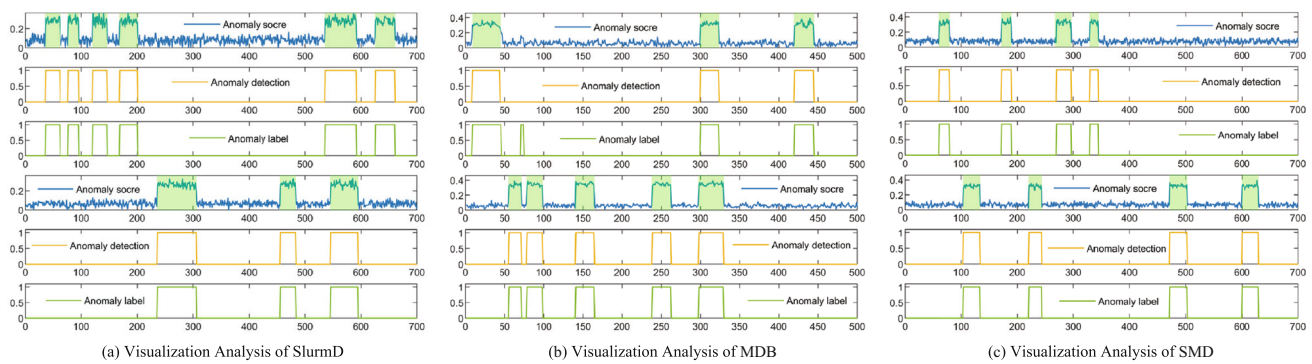


Fig. 15 Results of case analysis

In general, GCAD is less affected by parameters. This characteristic ensures its robustness in actual anomaly detection tasks, enhancing adaptability and reliability in complex environments.

Case study

In this study, we conducted an in-depth analysis of GCAD's anomaly detection performance on three distinct datasets. Specifically, we selected two sequence segments containing anomalies from each dataset and visualized the anomaly scores, detection results, and ground truth labels. The outcomes are presented in Fig. 15.

The visualization results indicate that GCAD can accurately identify the anomaly points in the sequences in most cases, showing a high degree of concordance with the ground truth labels. However, upon evaluating the MDB dataset, we identified a transient anomalous segment with a short duration that was not successfully detected as an anomaly. This occurrence may be attributed to the brief duration of the anomaly segment, resulting in insufficiently pronounced characteristics that impair the accuracy of anomaly detection. To address this issue, we can refine the detection strategy by adjusting the anomaly detection threshold or employing a more granular time window to enhance the recognition capability for transient anomalous segments. Overall, the proposed method demonstrates remarkable proficiency in capturing temporal patterns and associated dependencies, rendering it significantly effective for the task of anomaly detection.

Conclusion

This paper proposes a Graph-enhanced Learning model for Anomaly Detection in Cloud systems, namely GCAD, designed to efficiently identify performance anomalies in cloud systems. GCAD conducts data augmentation in both the time and frequency domains, generating diverse data

views. It utilizes GNNs and GRUs to capture the temporal dynamic features and topological structure information of cloud systems. Furthermore, we incorporate an encoder with a linear attention mechanism to further exploit the global features of hidden-layer embedding vectors. By adopting a contrastive learning strategy, the representation learning capability of the encoder is effectively enhanced. Experimental results demonstrate that the GCAD model outperforms existing benchmark models in cloud system performance anomaly detection tasks.

Reference [41] proposes an unsupervised learning framework based on graph neural networks. This framework involves an encoder that employs graph attention to convert complex network structures and node attributes into compact representations. Then, it utilizes a decoding strategy to reconstruct the network structure and node attributes. This strategy designs each decoding layer as the inverse process of the corresponding encoding layer to achieve precise reconstruction of the graph structure and reduce information loss. In this paper, we employ a graph attention mechanism to learn the representation of cloud system topologies. Experimental results demonstrate that this method can improve the integrity of information transmission. In future work, we intend to integrate the decoding strategy into GCAD to achieve higher efficiency and accuracy in the decoding phase. Moreover, we plan to incorporate more factors closely related to cloud system performance, such as task types, user groups, etc., and design targeted representation models to achieve a more comprehensive and accurate diagnosis of abnormal events.

Funding This research was supported by the National Key Research and Development Program (2018YFC1406200).

Data availability The authors confirm that the data supporting the conclusions of this study are available within the article. And all relevant data are within the paper.

Declarations

Conflict of interest The authors declare no conflict of interest.

Ethics approval and consent to participate Not applicable.

Consent for publication Consent has been granted by all authors and there is no conflict.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Barreto GPT, Zhou X, Liu L, Ding Z (2019) Bottleneck-aware task scheduling based on per-stage and multi-ml profiling. In: HPCC/SmartCity/DSS. IEEE, Zhangjiajie, pp 510–518
- Ibidunmoye O, Hernández-Rodríguez F, Elmroth E (2015) Performance anomaly detection and bottleneck identification. *ACM Comput Surv* 48(1):4:1–4:35
- Garg S, Kaur K, Kumar N, Kaddoum G, Zomaya AY, Ranjan R (2019) A hybrid deep learning-based model for anomaly detection in cloud datacenter networks. *IEEE Trans Netw Serv Manag* 16(3):924–935
- Soldani J, Brogi A (2023) Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: a survey. *ACM Comput Surv* 55(3):59:1–59:39
- Sauvanaud C, Kaāniche M, Kanoun K, Lazri K, Silvestre G (2018) Anomaly detection and diagnosis for cloud services: practical experiments and lessons learned. *J Syst Softw* 139:84–106
- Syed NAS, Sangeetha V, Prabhadevi C (2013) Entropy based anomaly detection system to prevent ddos attacks in cloud. *CoRR*. [arXiv:1308.6745](https://arxiv.org/abs/1308.6745)
- Farshchi M, Schneider J-G, Weber I, Grundy JC (2015) Experience report: Anomaly detection of cloud application operations using log and cloud metric correlation analysis. In: ISSRE. IEEE Computer Society, Gaithersbury, pp 24–34
- Dubey AK, Kumar A, García-Díaz V, Sharma AK, Kanhaiya K (2021) Study and analysis of sarima and lstm in forecasting time series data. *Sustain Energy Technol Assess* 47:101474
- Chimmula VKR, Zhang L (2020) Time series forecasting of COVID-19 transmission in Canada using lstm networks. *Chaos Solitons Fractals* 135:109864
- Livieris IE, Pintelas E, Pintelas PE (2020) A CNN-LSTM model for gold price time-series forecasting. *Neural Comput Appl* 32(23):17351–17360
- Hochenbaum J, Vallis OS, Kejariwal A (2017) Automatic anomaly detection in the cloud via statistical learning. *CoRR*. [arXiv:1704.07706](https://arxiv.org/abs/1704.07706)
- Zavrtanik V, Kristan M, Skočaj D (2021) Reconstruction by inpainting for visual anomaly detection. *Pattern Recognit* 112:107706
- Zavrtanik V, Kristan M, Skočaj D (2021) Draem-a discriminatively trained reconstruction embedding for surface anomaly detection. In: Proceedings of the IEEE/CVF international conference on computer vision. In: ICCV. IEEE, Montreal Canada, pp 8330–8339
- Gong D, Liu L, Le V, Saha B, Mansour MR, Venkatesh S, van den Hengel A (2019) Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection. In: ICCV. IEEE, Seoul, pp 1705–1714
- Vojir T, Sipka T, Aljundi R, Chumerin N, Reino DO, Matas J (2021) Road anomaly detection by partial image reconstruction with segmentation coupling. In: ICCV. IEEE, Montreal, pp 15631–15640
- Thill M, Konen W, Wang H, Bäck T (2021) Temporal convolutional autoencoder for unsupervised anomaly detection in time series. *Appl Soft Comput* 112:107751
- Jiao Y, Yang K, Song D, Tao D (2022) Timeautoad: autonomous anomaly detection with self-supervised contrastive loss for multivariate time series. *IEEE Trans Netw Sci Eng* 9(3):1604–1619
- Wang X, Qi G-J (2023) Contrastive learning with stronger augmentations. *IEEE Trans Pattern Anal Mach Intell* 45(5):5549–5560
- Chen T, Kornblith S, Norouzi M, Hinton GE (2020) A simple framework for contrastive learning of visual representations. In: ICML, volume 119 of Proceedings of Machine Learning Research. PMLR, Austria, pp 1597–1607
- Zheng M, Wang F, You S, Qian C, Zhang C, Wang X, Xu C (2021) Weakly supervised contrastive learning. In: ICCV. IEEE, Montreal, pp 10022–10031
- He Z, Chen P, Li X, Wang Y-F, Guangba Yu, Chen C, Li X, Zheng Z (2023) A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems. *IEEE Trans Neural Netw Learn Syst* 34(4):1705–1719
- Zhang Y, Chen Y, Wang J, Pan Z (2023) Unsupervised deep anomaly detection for multi-sensor time-series signals. *IEEE Trans Knowl Data Eng* 35(2):2118–2132
- Tang Y, Zhao L, Zhang S, Gong C, Li G, Yang J (2020) Integrating prediction and reconstruction for anomaly detection. *Pattern Recognit Lett* 129:123–130
- Amarbayasgalan T, Pham VH, Theera-Umpon N, Ryu KH (2020) Unsupervised anomaly detection approach for time-series in multi-domains using deep reconstruction error. *Symmetry* 12(8):1251
- Li Z, Zhao Y, Han J, Su Y, Jiao R, Wen X, Pei D (2021) Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding. In: SIGKDD. ACM, Singapore, pp 3220–3230
- Gao H, Qiu B, Durán RJ, Barroso WH, Yueshen X, Wang X (2023) TSMAE: a novel anomaly detection approach for internet of things time series data using memory-augmented autoencoder. *IEEE Trans Netw Sci Eng* 10(5):2978–2990
- Jiang T, Li Y, Xie W, Qian D (2020) Discriminative reconstruction constrained generative adversarial network for hyperspectral anomaly detection. *IEEE Trans Geosci Remote Sens* 58(7):4666–4679
- Jiahao Yu, Gao X, Zhai F, Li B, Xue B, Shiyuan F, Chen L, Meng Z (2024) An adversarial contrastive autoencoder for robust multivariate time series anomaly detection. *Expert Syst Appl* 245:123010
- Zhou H, Ke Yu, Zhang X, Guanlin W, Yazidi A (2022) Contrastive autoencoder for anomaly detection in multivariate time series. *Inf Sci* 610:266–280
- Kim HG, Kim S, Min S, Lee B (2023) Contrastive time-series anomaly detection. *IEEE Trans Knowl Data Eng* 36(10):5053–5065
- Miao J, Tao H, Xie H, Sun J, Cao J (2024) Reconstruction-based anomaly detection for multivariate time series using contrastive generative adversarial networks. *Inf Process Manag* 61(1):103569
- Jin M, Liu Y, Zheng Y, Chi L, Li Y-F, Pan S (2021) ANEMONE: graph anomaly detection with multi-scale contrastive learning. In: CIKM. ACM, Queensland, pp 3122–3126

33. Liu Y, Li Z, Pan S, Gong C, Zhou C, Karypis G (2022) Anomaly detection on attributed networks via contrastive self-supervised learning. *IEEE Trans Neural Netw Learn Syst* 33(6):2378–2392
34. Zheng Yu, Jin M, Liu Y, Chi L, Phan KT, Chen Y-PP (2023) Generative and contrastive self-supervised learning for graph anomaly detection. *IEEE Trans Knowl Data Eng* 35(12):12220–12233
35. Chen Z, Liu J, Su Y, Zhang H, Ling X, Lyu MR (2022) Adaptive performance anomaly detection for online service systems via pattern sketching. In: *ICSE*. ACM, Pittsburgh, pp 61–72
36. Llorido-Botran T, Huerta S, Tomás L, Tordsson J, Sanz B (2017) An unsupervised approach to online noisy-neighbor detection in cloud data centers. *Expert Syst Appl* 89:188–204
37. Shaifu G, Aroor DD (2020) Relevance feedback based online learning model for resource bottleneck prediction in cloud servers. *Neurocomputing* 402:307–322
38. Lyu L, Jin J, Rajasegarar S, He X, Palaniswami M (2017) Fog-empowered anomaly detection in iot using hyperellipsoidal clustering. *IEEE Internet Things J* 4(5):1174–1184
39. Farshchi M, Schneider J-G, Weber I, Grundy J (2018) Metric selection and anomaly detection for cloud operations using log and metric correlation analysis. *J Syst Softw* 137:531–549
40. Baek S, Kwon D, Kim J, Suh SC, Kim H, Kim I (2017) Unsupervised labeling for supervised anomaly detection in enterprise and cloud networks. In: *CSCloud*. IEEE Computer Society, New York, pp 205–210
41. Yang Y, Xiaorui S, Zhao B, Li GD, Pengwei H, Zhang J, Lun H (2024) Fuzzy-based deep attributed graph clustering. *IEEE Trans Fuzzy Syst* 32(4):1951–1964
42. Han D, Pan X, Han Y, Song S, Huang G (2023) Flatten transformer: Vision transformer using focused linear attention. In: *ICCV*. IEEE, Paris, pp 5938–5948
43. Yoo AB, Jette MA, Grondona M (2003) SLURM: simple linux utility for resource management. In: *JSSPP*, volume 2862 of *Lecture Notes in Computer Science*. Springer, Seattle, pp 44–60
44. Brazil B (2018) Prometheus: up & running: infrastructure and application performance monitoring. O'Reilly Media, Inc
45. Chakraborty M, Kundan AP (2021) Grafana. In: *Monitoring cloud-native applications: lead agile operations confidently using open source software*. Springer, pp 187–240
46. Naqvi SNZ, Yfantidou S, Zimányi E (2017) Time series databases and influxdb. *Stud Univ Libre Brux* 12:1–44
47. He Z. MBD dataset. <https://github.com/QAZASDEDC/TopoMAD>
48. Su Y, Zhao Y, Niu C, Liu R, Sun W, Pei D (2019) Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In: *SIGKDD*. ACM, Anchorage, pp 2828–2837
49. Breunig MM, Kriegel H-P, Ng RT, Sander J (2000) LOF: identifying density-based local outliers. In: Weidong C, Jeffrey FN, Philip AB (eds) *SIGMOD*. ACM, Dallas, pp 93–104
50. Tuli S, Casale G, Jennings NR (2022) Tranad: deep transformer networks for anomaly detection in multivariate time series data
51. Zhou B, Liu S, Hooi B, Cheng X, Ye J (2019) Beatgan: anomalous rhythm detection using adversarially generated time series. In: *IJCAI*. Macao, pp 4433–4439
52. Hundman K, Constantinou V, Laporte C, Colwell I, Söderström T (2018) Detecting spacecraft anomalies using lstms and non-parametric dynamic thresholding. In: *SIGKDD*. ACM, London, pp 387–395
53. Yang X, Zhang Z, Cui R (2022) Timeclr: a self-supervised contrastive learning framework for univariate time series representation. *Knowl Based Syst* 245:108606
54. Zhong G, Liu F, Jiang J, Wang B, Philip Chen CL (2024) Refining one-class representation: a unified transformer for unsupervised time-series anomaly detection. *Inf Sci* 656:119914

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.